

Mathematical Model of the Semantic Communication System

This document gives a formal description of the pipeline implemented in this repository. It defines every stage as a mathematical map, states the training objectives, and derives the signal-to-noise relationship the channel assumes. Symbols used here correspond directly to the code; file/line references are given where useful.

1 Notation and problem setup

We are given a labelled text-classification dataset

$$\mathcal{D} = \{(t^{(i)}, y^{(i)})\}_{i=1}^n, \quad y^{(i)} \in \{1, \dots, C\}, \quad C = 4,$$

where $t^{(i)}$ is a raw sentence and $y^{(i)}$ its AG-News class (World, Sports, Business, Sci/Tech).

The system is a composition of maps

$$t \xrightarrow{\text{BERT}} e \xrightarrow{f_\theta} z \xrightarrow{Q} q \xrightarrow{\text{channel}} r \xrightarrow{g_\phi} \hat{e} \xrightarrow{h_\psi} \ell \longrightarrow \hat{y},$$

with the constant dimensions

$$d_b = 768 \text{ (BERT)}, \quad D = \text{BOTTLENECK_DIM (even)}, \quad N = D/2 \text{ (\# I/Q symbols)}, \quad C = 4.$$

Trainable parameters are the autoencoder encoder θ , the autoencoder decoder ϕ , the classifier ψ , and — optionally — the constellation $\mathcal{C}$. BERT is **frozen**.

2 Semantic source encoder (frozen BERT)

The text is mapped to a fixed embedding by a frozen BERT model (`bert_encoder.py`). Pooling blends the [CLS] token of the last two hidden layers:

$$e = \frac{1}{2} h_{[\text{CLS}]}^{(L)} + \frac{1}{2} h_{[\text{CLS}]}^{(L-1)} \in \mathbb{R}^{d_b}, \quad d_b = 768.$$

Because BERT is frozen, e is precomputed once and cached; it is treated as the deterministic input to everything downstream.

3 Learned source encoder + power normalization

The AE encoder $f_\theta : \mathbb{R}^{768} \rightarrow \mathbb{R}^D$ compresses the embedding (`autoencoder.py:47`). Let $\tilde{z} = f_\theta(e)$. The output is then **L2-normalized to a fixed energy**:

$$z = \sqrt{NP} \frac{\tilde{z}}{\|\tilde{z}\|_2} \in \mathbb{R}^D, \quad P = \text{QAM_POWER},$$

so that the vector energy and the average per-symbol power are constants, independent of the input:

$$\|z\|_2^2 = NP \quad \implies \quad \frac{1}{N} \sum_{k=1}^N \|s_k\|_2^2 = P.$$

This fixed-power constraint is what makes the channel SNR (Section 6) well defined.

I/Q reshaping. The real vector is grouped into N two-dimensional symbols (one complex baseband symbol each):

$$z = (s_1, \dots, s_N), \quad s_k = (z_{2k-1}, z_{2k}) \in \mathbb{R}^2.$$

4 Constellation quantizer

4.1 Constellation

A square M -QAM constellation is a finite set of points (`quantizer.py:30`)

$$\mathcal{C} = \{c_1, \dots, c_M\} \subset \mathbb{R}^2, \quad M = \text{QAM_ORDER} = L^2, \quad L = \sqrt{M} \in \mathbb{N},$$

built from the symmetric PAM levels $\{-(L-1), \dots, -1, 1, \dots, (L-1)\}$ on each axis and scaled to unit average power:

$$\frac{1}{M} \sum_{j=1}^M \|c_j\|_2^2 = P.$$

By default $\mathcal{C}$ is fixed; if `LEARNED_CONSTELLATION = True` the points are parameters, re-projected onto the power constraint after every step.

4.2 Soft and hard assignment

For each symbol s_k define the squared distances to all constellation points

$$d_{k,j} = \|s_k - c_j\|_2^2.$$

The **soft assignment** is a temperature-weighted softmax (`quantizer.py:127`) with hardness $\sigma_q > 0$:

$$w_{k,j} = \frac{\exp(-\sigma_q d_{k,j})}{\sum_{l=1}^M \exp(-\sigma_q d_{k,l})}, \quad s_k^{\text{soft}} = \sum_{j=1}^M w_{k,j} c_j.$$

The **hard assignment** snaps to the nearest point:

$$j_k^* = \arg \min_j d_{k,j}, \quad s_k^{\text{hard}} = c_{j_k^*}.$$

4.3 Straight-through estimator

The transmitted symbol is the hard one, but the gradient is taken from the soft one, via the straight-through construction (`quantizer.py:135`):

$$q_k = s_k^{\text{hard}} + (s_k^{\text{soft}} - \text{sg}[s_k^{\text{soft}}]),$$

where $\text{sg}[\cdot]$ is the stop-gradient operator ($\text{sg}[u] = u$ in the forward pass, $\nabla \text{sg}[u] = 0$ in the backward pass). Hence

$$\underbrace{q_k = s_k^{\text{hard}}}_{\text{forward}}, \quad \underbrace{\frac{\partial q_k}{\partial s_k} = \frac{\partial s_k^{\text{soft}}}{\partial s_k}}_{\text{backward}}.$$

The full quantizer map is $q = Q(z) = (q_1, \dots, q_N) \in \mathbb{R}^D$.

4.4 Hardness annealing

The hardness is increased once per optimizer step t (`quantizer.py:92`):

$$\sigma_q(t) = \min(\sigma_{\max}, \sigma_0 + \rho t), \quad \sigma_0 = \text{SOFT_Q_INIT}, \quad \rho = \text{SOFT_Q_ANNEAL_RATE}, \quad \sigma_{\max} = \text{SOFT_Q_MAX}.$$

As $\sigma_q \rightarrow \infty$ the softmax converges to the argmin, so $s_k^{\text{soft}} \rightarrow s_k^{\text{hard}}$ and the training-time (soft) behavior matches the deployed (hard) behavior.

4.5 Constellation-usage regularizer

The empirical usage distribution over constellation points, averaged over a batch of size B (`quantizer.py:137`), is

$$\hat{P}(c_j) = \frac{1}{BN} \sum_{b=1}^B \sum_{k=1}^N w_{k,j}^{(b)}, \quad \sum_{j=1}^M \hat{P}(c_j) = 1.$$

Its divergence from the uniform distribution $U(c_j) = 1/M$ is the KL term (`quantizer.py:153`)

$$D_{\text{KL}}(\hat{P} \parallel U) = \sum_{j=1}^M \hat{P}(c_j) \log \frac{\hat{P}(c_j)}{1/M} = \log M - H(\hat{P}),$$

which is minimized (equal to 0) exactly when all points are used equally. This penalizes constellation mode-collapse.

5 Channel

Two channel models are used (`channel.py`).

Identity (clean) channel: $r = q$.

AWGN channel: additive white Gaussian noise applied independently to every real coordinate,

$$r = q + n, \quad n \sim \mathcal{N}(0, \sigma_n^2 I_D), \quad \sigma_n = \text{NOISE_STD}.$$

Per symbol, $r_k = q_k + n_k$ with $n_k \sim \mathcal{N}(0, \sigma_n^2 I_2)$.

6 Signal-to-noise ratio (derivation)

Each transmitted symbol has average power $\mathbb{E}\|q_k\|_2^2 = P$ by the normalization of Sections 3–4. The noise power per symbol is $\mathbb{E}\|n_k\|_2^2 = 2\sigma_n^2$ over two real dimensions, i.e. the per-real-dimension signal and noise powers are $P/2$ and σ_n^2 . The signal model therefore has a fixed SNR

$$\text{SNR} = \frac{P}{\sigma_n^2}, \quad \text{SNR}_{\text{dB}} = 10 \log_{10} \frac{P}{\sigma_n^2}.$$

With $P = 1$: $\sigma_n = 0.1 \Rightarrow 20$ dB, $\sigma_n = 0.3 \Rightarrow \approx 10.5$ dB, $\sigma_n = 0.5 \Rightarrow \approx 6$ dB. The fixed-power normalization is precisely what makes this mapping from σ_n to SNR independent of the input.

7 Learned decoder and classifier

AE decoder $g_\phi : \mathbb{R}^D \rightarrow \mathbb{R}^{768}$ reconstructs the BERT embedding from the received signal (`autoencoder.py:56`):

$$\hat{e} = g_\phi(r) \in \mathbb{R}^{768}.$$

Task classifier $h_\psi : \mathbb{R}^{768} \rightarrow \mathbb{R}^C$ is a residual MLP (`decoder.py`) that reads the *reconstruction* $\hat{e}$ (not the bottleneck) and outputs logits

$$\ell = h_\psi(\hat{e}) \in \mathbb{R}^C, \quad \hat{y} = \arg \max_{c \in \{1, \dots, C\}} \ell_c, \quad p(c | t) = \text{softmax}(\ell)_c.$$

8 Training objectives (two separate phases)

Training is **not** end-to-end; the autoencoder and the classifier are optimized in two sequential phases (`train.py`).

Phase 1 — unsupervised compression

Optimize the encoder, decoder (and constellation, if learnable) with **no labels**, minimizing reconstruction MSE plus the usage regularizer (`train.py:104`):

$$\mathcal{L}_1(\theta, \phi) = \mathbb{E}_e \left[\|\hat{e} - e\|_2^2 \right] + \lambda_{\text{KL}} D_{\text{KL}}(\hat{P} \| U), \quad \lambda_{\text{KL}} = \text{KL_LAMBDA},$$

where $\hat{e} = g_\phi(\text{channel}(Q(f_\theta(e))))$ carries the straight-through gradient through Q . Model selection uses validation MSE (early stopping).

Note: the KL term is minimized jointly with MSE and contributes a genuine gradient through the graph-connected usage tensor (see `quantizer.py kl_to_uniform`).

Phase 2 — supervised classification

Freeze (θ, ϕ) and train only the classifier ψ on the (frozen) reconstructions, minimizing class-weighted, label-smoothed cross-entropy (`train.py:142`):

$$\mathcal{L}_2(\psi) = \mathbb{E}_{(e, y)} \left[- \sum_{c=1}^C \alpha_c \tilde{y}_c \log \text{softmax}(\ell)_c \right],$$

with class weights $\alpha = \text{CLASS_WEIGHTS}$ and label-smoothed targets

$$\tilde{y}_c = (1 - \varepsilon)\mathbb{I}[c = y] + \frac{\varepsilon}{C}, \quad \varepsilon = \text{LABEL_SMOOTHING}.$$

Because the Phase-2 forward pass runs under a no-gradient context for the AE, no classification gradient reaches θ , ϕ , or $\mathcal{C}$. Model selection uses validation accuracy.

9 Separation variant: LDPC channel coding (inference only)

The “separate source–channel coding” configuration replaces the differentiable quantizer path at **test time** with hard symbol indices protected by a rate- r LDPC code (`ldpc_codec.py`, `pipeline.py:56`). Let the hard indices be

$$j_k^\star = \arg \min_j \|s_k - c_j\|_2^2 \in \{0, \dots, M - 1\}.$$

Encode path (info bits $\rightarrow$ transmitted symbols):

$$\mathbf{u} = \text{Gray}(j^\star) \in \{0, 1\}^K \xrightarrow{\text{encode}} \mathbf{x} = G^\top \mathbf{u} \bmod 2 \in \{0, 1\}^n \xrightarrow{\text{map}} \text{QAM symbols}, \quad r = \frac{K}{n} = \text{LDPC_CODE_RAT}$$

where G is the LDPC generator matrix over $\text{GF}(2)$. After the AWGN channel, decode path (received $\rightarrow$ info bits) uses soft demapping to log-likelihood ratios followed by belief propagation with parity-check matrix H :

$$\Lambda_m = \log \frac{\Pr(x_m = 0 \mid r)}{\Pr(x_m = 1 \mid r)}, \quad \hat{\mathbf{u}} = \text{BP}_H(\Lambda), \quad H\hat{\mathbf{x}} = \mathbf{0} \bmod 2.$$

The recovered symbols feed the AE decoder g_ϕ exactly as in Section 7. A rate-1/2 code doubles the transmitted bits, so with $D = 64$, $M = 16$: 32 symbols $\rightarrow$ 128 info bits $\rightarrow$ 256 coded bits $\rightarrow$ 64 symbols $\rightarrow$ 128 transmitted real dimensions, matching the joint model’s budget.

10 End-to-end summary

Putting the differentiable path together, the trained system computes

$$\hat{y}(t) = \arg \max h_\psi \left(g_\phi \left(Q_{\sigma_q} \left(f_\theta (\text{BERT}(t)) \right) + n \right) \right), \quad n \sim \mathcal{N}(0, \sigma_n^2 I_D),$$

trained by

$$(\theta^\star, \phi^\star) = \arg \min_{\theta, \phi} \mathcal{L}_1, \quad \psi^\star = \arg \min_{\psi} \mathcal{L}_2|_{\theta^\star, \phi^\star}.$$

The central experimental question is the **source–channel separation** trade-off: whether one jointly-learned noise-robust encoder (large D , trained with n) beats a clean-trained compressor (small D) protected by a classical LDPC code, as a function of the channel SNR $10 \log_{10}(P/\sigma_n^2)$.

11 Symbol reference

Symbol	Meaning	Code name
$d_b = 768$	BERT embedding dimension	BERT_DIM
D	bottleneck dimension (real)	BOTTLENECK_DIM
$N = D/2$	number of I/Q symbols	num_symbols
$C = 4$	number of classes	NUM_CLASSES
M	constellation order	QAM_ORDER
P	average symbol power	QAM_POWER
σ_q	quantizer hardness	sigma_q
$\sigma_0, \rho, \sigma_{\max}$	annealing schedule	SOFT_Q_INIT, SOFT_Q_ANNEAL_RATE, SOFT_Q_MAX
λ_{KL}	usage-regularizer weight	KL_LAMBDA
σ_n	AWGN standard deviation	NOISE_STD
ε	label smoothing	LABEL_SMOOTHING
α_c	class weights	CLASS_WEIGHTS
$r = K/n$	LDPC code rate	LDPC_CODE_RATE
f_θ, g_ϕ, h_ψ	AE encoder, AE decoder, classifier	AEEncoder, AEDecoder, TaskDecoder